

**PROGRAMOWANIE GIER
W UNITY I C#
WYKŁAD 6 –
INTELIGENCJA W GRACH
ZAPIS/ODCZYT DANYCH**

Bogumiła Hnatkowska

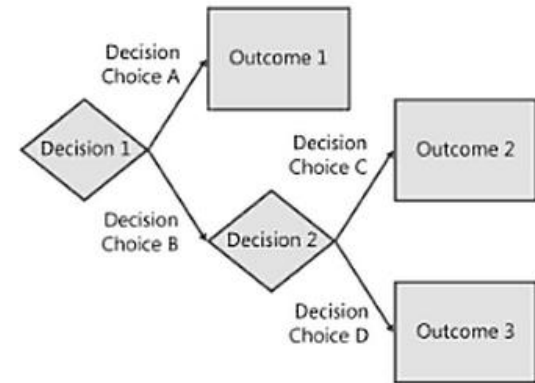
SZTUCZNA INTELIGENCJA W GRACH

- Techniki podejmowania decyzji:
 - Drzewa decyzyjne
 - Drzewa zachowań
 - Maszyny stanów
 - Planowanie akcji zorientowane na cel (Goal Oriented Action Planning, GOAP)
 - Logika rozmyta
 - Sieci neuronowe
 - Algorytmy genetyczne
 - ...



DRZEWA DECYZYJNE

- Do modelowania złożonych decyzji
- Składowe:
 - Węzły decyzyjne
 - Decyzje (liście)
 - Etykietowane przejścia między węzłami (wyniki decyzji)
- Należy podjąć decyzje, kiedy drzewo ma być sprawdzane, np. w każdej ramce, co 1s, po zakończeniu akcji
- Agent za każdym razem przechodzi przez całe drzewo, aż znajdzie akcję do wykonania
- Możliwe implementacje:
 - Sekwencja instrukcji „if”
 - Dedykowane klasy reprezentujące węzły



DRZEWA DECYZYJNE

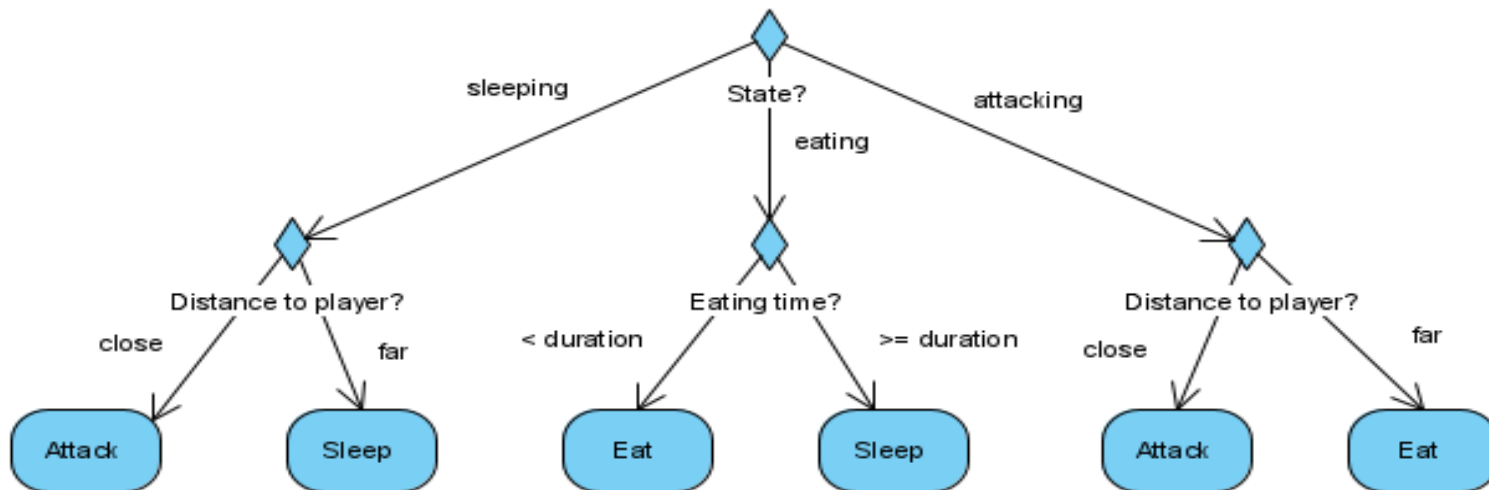
- Zalety:
 - Modularność
 - Hierarchiczność
 - Zrozumiałość
- Wady:
 - Brak informacji od węzłów na temat przetwarzania; trudna obsługa potencjalnych błędów

KieranCoppins/Decision-Tree-Package: An open source Unity package for creating Decision Trees with a visual editor. The package comes with some samples to help teach users how to use it and an action manager to handle executing the results of the decision tree. ([github.com](https://github.com/KieranCoppins/Decision-Tree-Package))

DRZEWA DECYZYJNE, PRZYKŁAD

- Drzewo sprawdzane podczas funkcji Update, co pewien określony czas, np. 1s
- Agent wykonuje (uruchamia) akcję tylko, gdy jest inna od dotychczas wykonywanej
- Agent śpi, dopóki gracz nie podejdzie odpowiednio blisko – wtedy go atakuje; po ataku (gdy gracz odejdzie) musi zjeść i je przez określony czas (nie można tej czynności przerwać), po czym wraca do spania

Visual Paradigm Standard (Bożena Institute of Informatics))



DRZEWA DECYZYJNE, PRZYKŁAD C.D.

```
public interface IDecision {  
    Odwołania: 4  
    string Evaluate(IAgent agent);  
}
```

1 odwołanie

```
public class DecisionNode : IDecision {  
    Odwołania: 4  
    public string Title { get; set; }  
    Odwołania: 5  
    public Dictionary<string, IDecision> Branches { get; set; } // nazwa krawędzi, decyzja  
    Odwołania: 5  
    public Func<IAgent, string> Test { get; set; } // agent, nazwa krawędzi  
    Odwołania: 4  
    public string Evaluate(IAgent agent) {  
        string result = Test(agent);  
  
        IDecision decision;  
        if (Branches.TryGetValue(result, out decision))  
            return decision.Evaluate(agent);  
        return "";  
    }  
}
```

DRZEWA DECYZYJNE, PRZYKŁAD C.D.

```
Odwołania: 0
public class ActionNode : IDecision {
    Odwołania: 7
    public string Action { get; set; }
    Odwołania: 4
    public string Evaluate(IAgent agent){
        return Action;
    }
}
```

```
Odwołania: 0
public interface IAgent {
    Odwołania: 3
    string getDistanceToPlayer(); // close, far
    Odwołania: 2
    string getState(); // eating, attacking, sleeping
    Odwołania: 2
    string isEatingOver(); // yes, no
}
```



DRZEWA DECYZYJNE, PRZYKŁAD C.D.

```
private static DecisionNode MainDecisionTree()
{
    // Decision 2
    var distanceFromPlayer1 = new DecisionNode {
        Title = "Distance from player when sleeping",
        Test = (agent) => agent.GetDistanceToPlayer(),
        Branches = new Dictionary<string, IDecision>() {
            { "close", new ActionNode() { Action = "Attack" } },
            { "far", new ActionNode() { Action = "Sleep" } }
        }
    };

    // Decision 1
    var root = new DecisionNode
    {
        Title = "root",
        Test = (agent) => agent.GetState(),
        Branches = new Dictionary<string, IDecision>() {
            { "Sleep", distanceFromPlayer1 },
            { "Eat", eatingTime },
            { "Attack", distanceFromPlayer2 }
        }
    };

    return root;
}
```

...



DRZEWA DECYZYJNE, PRZYKŁAD C.D.

```
void Update() {  
    if (startTime == 0) {  
        startTime = Time.time;  
  
        var action = root.Evaluate(agent);  
        Debug.Log($"DECISION TREE: " + action);  
  
        if (action != current) {  
            current = action;  
            perform(current);  
        }  
    }  
  
    if (Time.time - startTime > updateDuration)  
        startTime = 0;  
}
```



DRZEWA ZACHOWAŃ

- Drzewo składające się z węzłów różnych typów:
 - Korzenia
 - Węzłów sterujących – mają co najmniej 1 dziecko
 - Węzłów wykonawczych (akcje, warunki) – liście
- Korzeń drzewa generuje sygnały (ticks) w równych odstępach czasu, które są wysyłane do jego dzieci
- Węzły wykonawcze zwracają stan:
 - Success – operacja zakończona sukcesem
 - Failure – operacja zakończona niepowodzeniem
 - Running – operacja w toku
- Węzły sterujące (wysyłają sygnały od lewej do prawej):
 - Sekwencja (sequence, ->) – zwraca sukces, jeżeli wszystkie węzły zwracają sukces, Running jeżeli przetwarzane dziecko zwraca Running, Failure w przeciwnym przypadku
 - Selector (selector, fallback, ?) – zwraca Running, jeżeli przetwarzane dziecko zwraca Running; sukces, jeżeli przetwarzane dziecko zwraca sukces; Failure w przeciwnym przypadku
 - Decorator (romby z nazwą) – np. Inverter
 - Zrównoległacz (Parallel) z N węzłami (przy progu sukcesu M) – zwraca sukces jeżeli M węzłów zwróci sukces, niepowodzenie, jeżeli N-M zwróci niepowodzenie, Running w przeciwnym przypadku

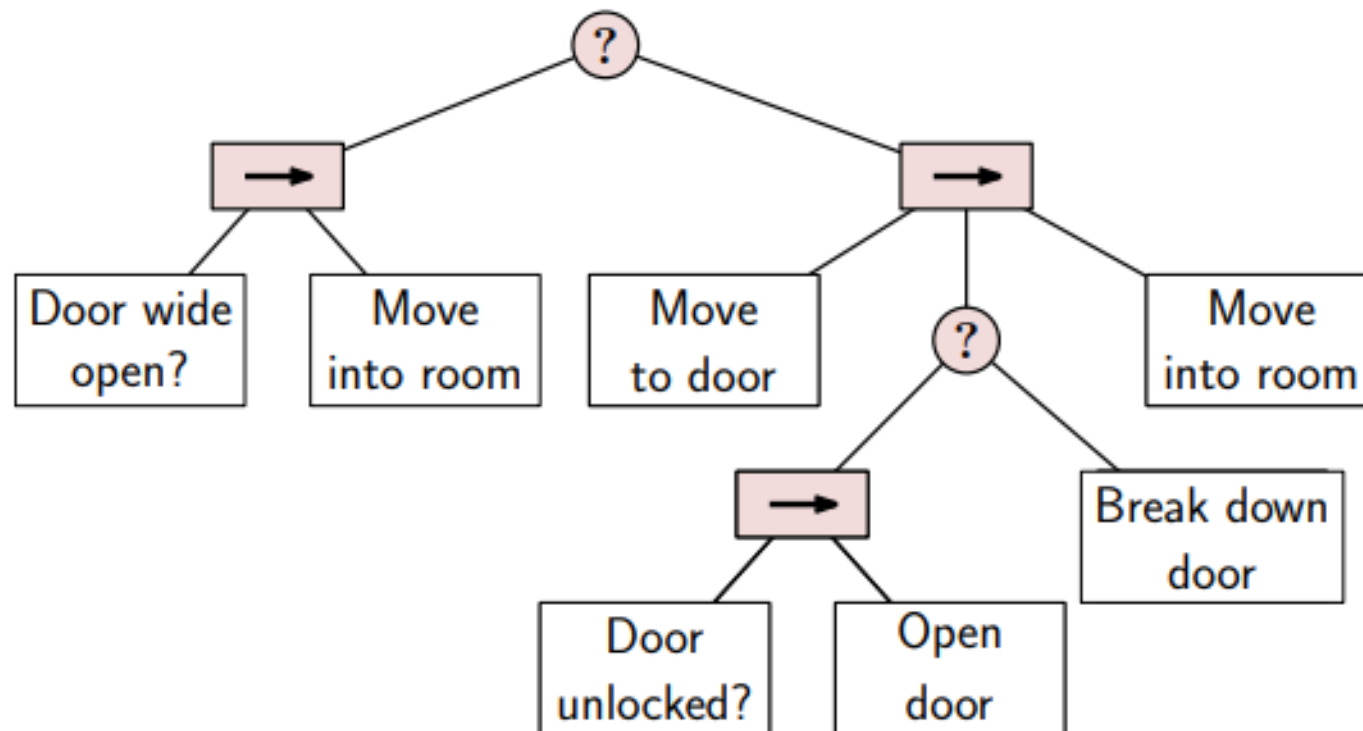


DRZEWA ZACHOWAŃ

- Zalety:
 - Modularność i ponowne użycie
 - Hierarchiczna organizacja
 - Zrozumiałość
- Wady:
 - Trudność implementacji
 - Możliwa obniżona wydajność
- Przykładowa implementacja:
<https://github.com/PacktPublishing/Unity-2017-Game-AI-Programming-Third-Edition/tree/master/Chapter06> -
repozytorium książki [Unity 2017 Game AI Programming Third Edition](#)



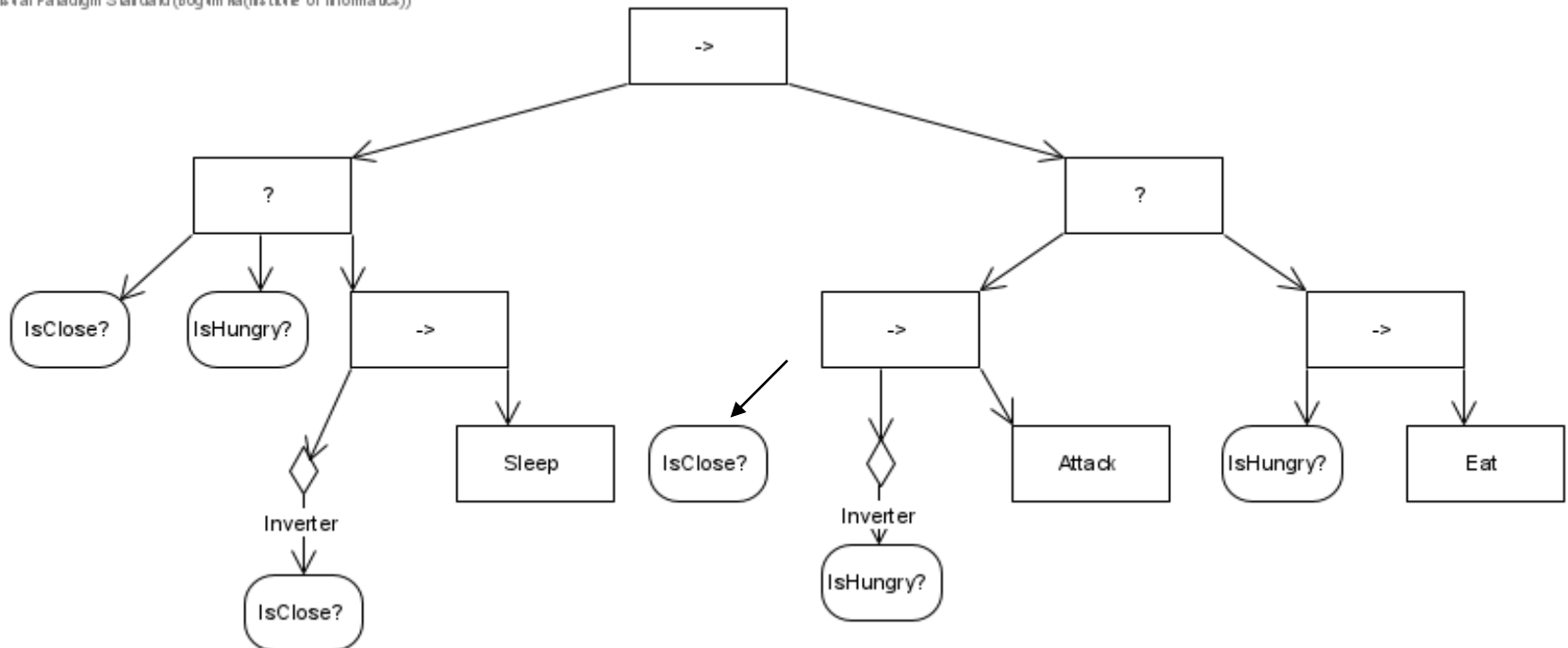
DRZEWA ZACHOWAŃ, PRZYKŁAD



DRZEWA ZACHOWAŃ, PRZYKŁAD

- Drzewo bez pamięci (za każdym razem przechodzimy przez drzewo od początku; nie zapamiętujemy statusów wykonania)

Visual Paradigm Standard (Bogum Ha(Institute of Informatics))



DRZEWA ZACHOWAŃ, PRZYKŁAD C.D.

1 odwołanie

```
private Sequence MainBehaviouralTree()
{
    var isCloseAction = new Nodes.ActionNode(isClose);
    var isHungryAction = new Nodes.ActionNode(isHungry);
    var isCloseInverter = new Inverter(isCloseAction);
    var isHungryInverter = new Inverter(isHungryAction);
    var sleepAction = new Nodes.ActionNode(sleep);
    var attackAction = new Nodes.ActionNode(attack);
    var eatAction = new Nodes.ActionNode(eat);

    var sleepSeq = new Sequence(new List<Node>() { isCloseInverter, sleepAction });
    var firstSel = new Selector(new List<Node>() { isCloseAction, isHungryAction, sleepSeq });

    var attackSeq = new Sequence(new List<Node>() { isCloseAction, isHungryInverter, attackAction });
    var eatSeq = new Sequence(new List<Node>() { isHungryAction, eatAction });
    var secondSel = new Selector(new List<Node>() { attackSeq, eatSeq });

    var root = new Sequence(new List<Node>() { firstSel, secondSel });
    return root;
}
```



DRZEWA ZACHOWAŃ, PRZYKŁAD C.D.

```
private bool hasToEat = false;
```

1 odwołanie

```
IEnumerator EatingTime()  
{  
    yield return new WaitForSeconds(eatingDuration);  
    hasToEat = false;  
}
```

1 odwołanie

```
NodeStates isHungry()  
{  
    return (hasToEat) ? NodeStates.SUCCESS : NodeStates.FAILURE;  
}
```

1 odwołanie

```
NodeStates sleep() {  
    if (action == "Attack")  
        hasToEat = true;  
  
    action = "Sleep";  
  
    return NodeStates.RUNNING;  
}
```



DRZEWA ZACHOWAŃ, PRZYKŁAD C.D.

```
NodeStates attack() {  
    if (action != "Attack")  
        animator.SetTrigger("Attack");  
  
    action = "Attack";  
    return NodeStates.RUNNING;  
}
```

1 odwołanie

```
NodeStates eat() {  
    if (action != "Eat")  
    {  
        animator.SetTrigger("Eat");  
        StartCoroutine(EatingTime());  
        action = "Eat";  
    }  
    else  
    {  
        if (isEatingOver())  
            return NodeStates.SUCCESS;  
    }  
}
```

```
void Update() {  
    if (startTime == 0) {  
        startTime = Time.time;  
        root.Evaluate(); // wywołanie BT  
    }  
  
    if (Time.time - startTime > updateDuration) {  
        startTime = 0;  
    }  
}
```



MASZYNY STANÓW (FINITE STATE MACHINES, FSM)

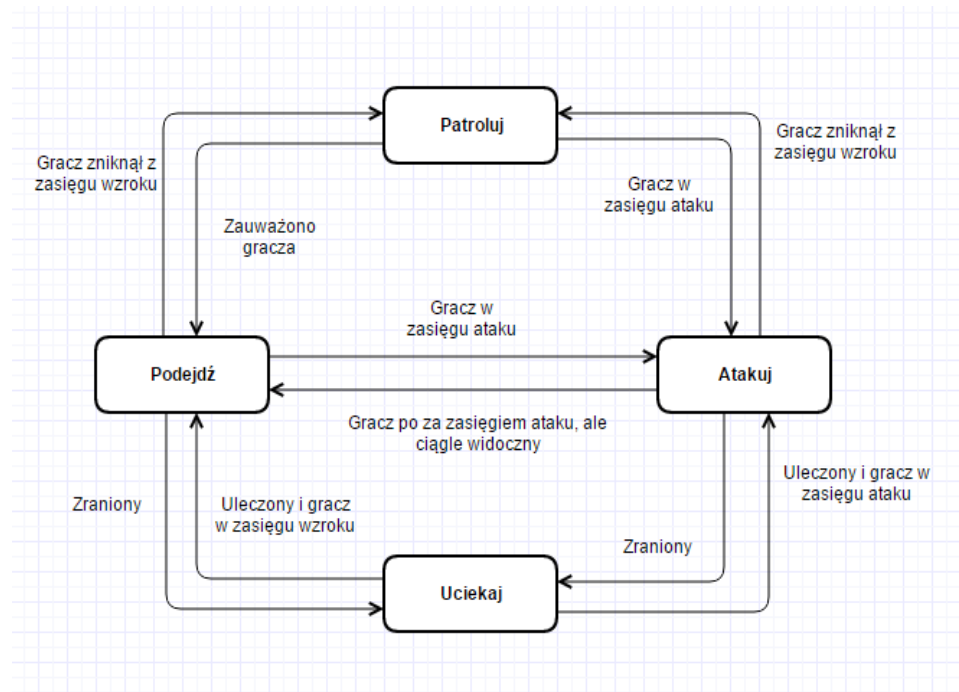
- Podstawowe elementy:

- Węzły (akcje)
- Tranzycje/przejścia (warunki)

- W danym obiekcie obiekt tylko w 1 stanie

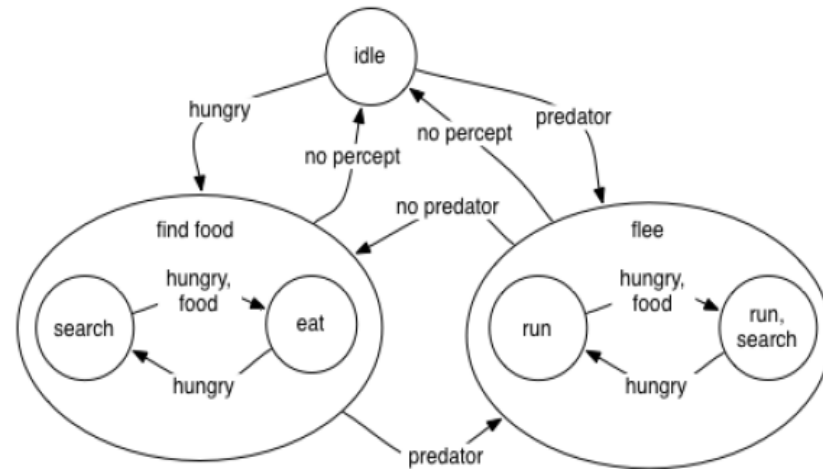
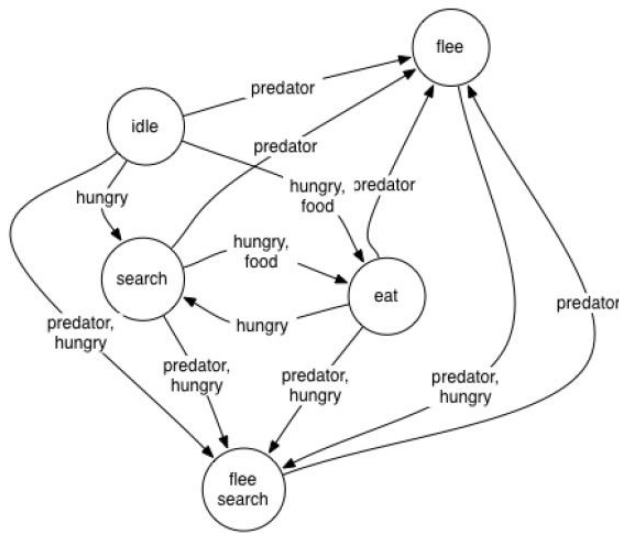
- Możliwość zdefiniowania dodatkowych akcji wykonywanych przy:

- Wejściu
- Wyjściu
- W stanie



HIERARCHICZNE MASZYNY STANÓW (HFSM)

- Stany mogą być reprezentowane przez osobne maszyny
- Stan obiektu jest reprezentowany przez listę stanów, kolejne elementy listy reprezentują stany złożone, np. (flee, run)
- Przy wchodzeniu/wychodzeniu ze stanu złożonego akcje wykonywane są w odpowiedniej kolejności:
 - Enter Flee, Enter Run
 - Exit Run, Exit Flee



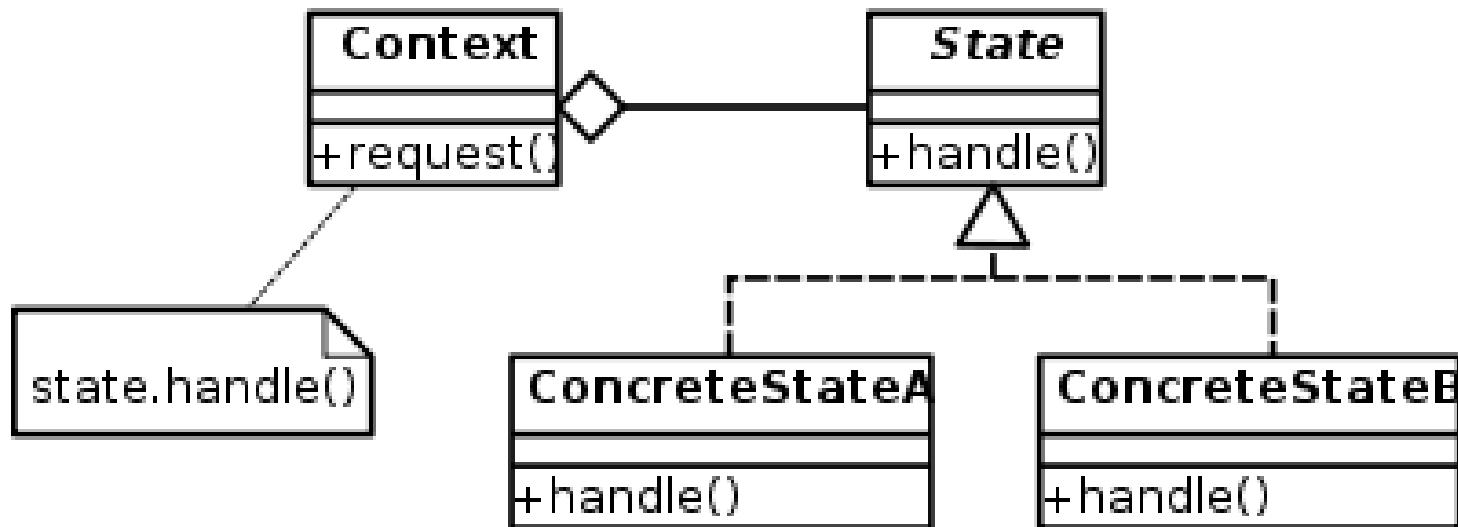
MASZYNY STANÓW

- Zalety:
 - Łatwe do implementacji
 - Zrozumiałe
- Wady:
 - Brak możliwości łatwego wykorzystania fragmentów (chyba, że mamy do czynienia z HFSM)
 - Rosnąca złożoność wraz z liczbą stanów/tranzycji i malejąca pielęgnowalność



ZARZĄDZANIE STANEM OBIEKTÓW

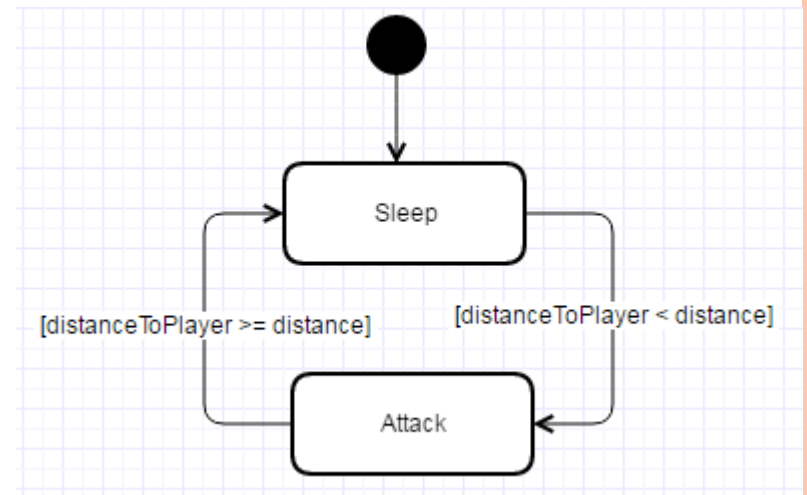
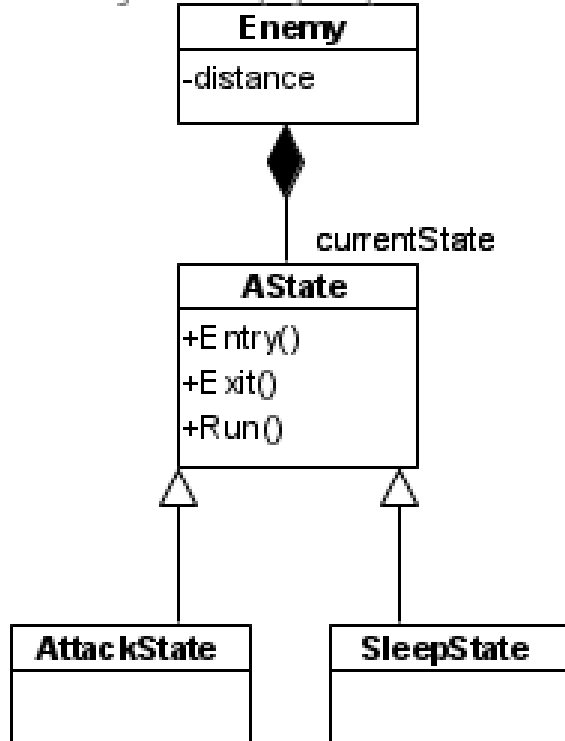
- Rozwiązanie oparte o wzorzec Stan



ZARZĄDZANIE STANEM OBIEKTÓW, C.D.

- Rozwiązanie oparte o wzorzec Stan, c.d.

Visual Paradigm Standard (Boqun He/Institute of Informatics)



Maszyna stanów Enemy



ZARZĄDZANIE STANEM OBIEKTÓW, C.D.

```
public abstract class AState {  
    protected Enemy enemy;          // state machine owner  
    Odwołania: 2  
    public AState(Enemy enemy) {  
        this.enemy = enemy;  
    }  
    Odwołania: 4  
    public virtual void Entry() { }  
    Odwołania: 3  
    public virtual void Run() { }  
    1 odwołanie  
    public virtual void Exit() { }  
    Odwołania: 2  
    public void MoveTo(AState newState) {  
        Exit();  
        newState.Entry();  
        enemy.ChangeState(newState);  
    }  
}
```



ZARZĄDZANIE STANEM OBIEKTÓW, C.D.

```
public class SleepState: AState {  
    private float distance;  
    1 odwołanie  
    public SleepState(Enemy enemy, float distance) : base(enemy) {  
        this.distance = distance;  
    }  
    Odwołania: 3  
    public override void Entry() {  
        Animator animator = enemy.GetComponent<Animator>();  
        animator.SetTrigger("sleep");  
    }  
    Odwołania: 2  
    public override void Run(){  
        var direction = (Camera.main.transform.position  
                        - enemy.transform.position).normalized;  
        Ray ray = new Ray(enemy.transform.position, direction);  
  
        if (Physics.Raycast(ray, distance)) {  
            MoveTo(new AttackState(enemy, distance));  
        }  
    }  
}
```



ZARZĄDZANIE STANEM OBIEKTÓW, C.D.

○ Rozwiązanie oparte o wzorzec Stan, c.d.

```
public class Enemy : MonoBehaviour {  
    public float distance = 1.0f;  
    private AState currentState;  
  
    Unity Message | Odwołania: 0  
    void Awake () {  
        currentState = new AttackState(this, distance);  
        currentState.Entry();  
    }  
  
    Unity Message | Odwołania: 0  
    void Update () {  
        currentState.Run();  
    }  
  
    1 odwołanie  
    public void ChangeState(AState newState) {  
        currentState = newState;  
    }  
}
```



PLANOWANIE AKCJI W OPARCIU O CEL

- Elementy składowe:

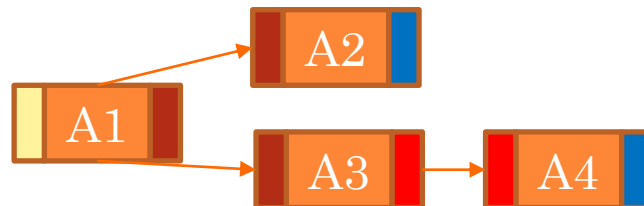
- Agent – chce zrealizować cel wykonując akcje; cel może być opisany jako szczególny warunek końcowy

- Akcje (niepowiązane ze sobą):

- Warunki wstępne
- Warunki końcowe (efekty)
- Koszt
- [Proceduralne warunki wstępne]

- Planner – buduje sekwencje akcji (na podstawie ich warunków wstępnych/końcowych), aby zrealizować cel agenta; wybiera najtańszą sekwencję do realizacji lub ustala, że nie da się zbudować planu

- Do niebieskiego celu od początkowego żółtego:



PLANOWANIE AKCJI W OPARCIU O CEL

- Stan lokalny (agenta), globalny (świata) – zbiór spełnionych warunków (faktów)
- Prosta maszyna stanów (zwykle 2-3 stany), związana z agentem
 - Idle → MoveTo → Perform → [MoveTo] → Perform → ... Idle
 - Idle → Perform → Perform → ... → Idle
 - W stanie Idle obliczany jest aktualny plan dla agenta
 - Przejście do kolejnego stanu tylko, gdy plan istnieje
 - W stanie Perform – wykonywane są kolejne akcje zgodnie z planem; jeżeli trzeba, przechodzimy przez stan MoveTo
 - MoveTo – dodatkowa akcja polegająca na przemieszczeniu agenta do celu
- Tutoriale GOAP
 - <https://learn.unity.com/course/artificial-intelligence-for-beginners>
 - <https://gamedevelopment.tutsplus.com/tutorials/goal-oriented-action-planning-for-a-smarter-ai--cms-20793>



GOAP – PRZYKŁAD

○ Akcje:

- Atak: pre [brak]; post [hasAttacked] – tak długo, jak gracz w zasięgu, koszt 1, warunek proceduralny
- Jedzenie: pre [hasAttacked]; post [isGlad] – określony czas, np. 4 sekundy (nie da się przerwać), koszt 1
- Spanie: pre [brak]; post [isGlad] – tylko, gdy gracz poza zasięgiem, koszt 1

○ Agent:

- Cel: isGlad
- Brak stanu lokalnego

○ Plany:

- Gdy gracz w zasięgu agenta: Atak -> Jedzenie, koszt 2
- Gdy gracz poza zasięgiem agenta: Spanie, koszt 1 (ale są warunki proceduralne)



GOAP – PRZYKŁAD (DEFINICJA AKCJI)

```
public class SleepingAction : GoapAction
```

```
{
```

Unity Message | Odwołania: 0

```
public new void Start()
```

```
{
```

```
    base.Start();
```

```
    action_name = "SleepingAction";
```

```
    addEffect("isGlad", true);
```

```
}
```

Odwołania: 2

```
public override bool isDone()...
```

Odwołania: 2

```
public override bool checkProceduralPrecondition(GameObject agent)...
```

Odwołania: 2

```
public override bool perform(GameObject agent)
```

```
{
```

```
    Debug.Log("Sleeping action run");
```

```
    Animator animator = agent.GetComponent<Animator>();
```

```
    animator.Play("Sleep");
```

```
    return true;
```

```
}
```



GOAP – PRZYKŁAD (DEFINICJA CELU DLA AGENTA)

↳ Ony jest operacją Enemy, Generowanie

```
public class Enemy : MonoBehaviour, IGoap
```

```
{
```

```
    /**
```

```
     * Key-Value data that will feed the GOAP actions and system while planning.
```

```
    */
```

Odwołania: 3

```
    public HashSet<KeyValuePair<string, object>> getWorldState()...
```

```
    /**
```

```
     * Implement in subclasses
```

```
    */
```

Odwołania: 7

```
    public HashSet<KeyValuePair<string, object>> createGoalState() {
```

```
        HashSet<KeyValuePair<string, object>> goal = new HashSet<KeyValuePair<string, object>>();
```

```
        goal.Add(new KeyValuePair<string, object>("isGlad", true));
```

```
        return goal;
```

```
    }
```



LOGIKA ROZMYTA

- **Logika rozmyta** (ang. Fuzzy logic) – logika wielowartościowa zaproponowana przez Lotfi Zadeha w 1965 r. Znajduje zastosowanie w systemach sterowania czy w budowie systemów ekspertowych w przypadku nieostrych reguł wnioskowania.
- **Zbiór rozmyty** – zbiór, do którego przynależność określona jest **funkcją przynależności (stopień przynależności)**; funkcja może przyjmować wartości z przedziału $[0, 1]$; może przyjąć postać trójkąta, trapezu, rozkładu gaussa etc.
- **Zmienna lingwistyczna** – zmienna, reprezentująca pewną cechę lub właściwość (np. temperatura), której wartości chcemy opisać zbiorami rozmytymi; każdy zbiór rozmyty jest reprezentowany etykietą (np. temperatura niska, temperatura średnia) i zdefiniowany funkcją przynależności



LOGIKA ROZMYTA, C.D.

- Przykład na podstawie: [jFuzzyLogic \(sourceforge.net\)](https://sourceforge.net/projects/jfuzzylogic/)
- Wysokość napiwka ustalimy na podstawie jakości jedzenia i jakości usługi (zmienne lingwistyczne) za pomocą reguł wnioskowania
- Proces:
 - Definicja zmiennych lingwistycznych i ich reprezentacji (fuzyfikacja)
 - Wnioskowanie z wykorzystaniem reguł
 - Obliczenie wyniku (defuzyfikacja), na podstawie podanych parametrów

Przykładowa implementacja w C#

[robertrumney/fuzzy-logic: Unity C# fuzzy logic library for a wide variety of fuzzy tasks \(github.com\)](https://github.com/robertrumney/fuzzy-logic)



LOGIKA ROZMYTA, C.D.

○ Definicja zmiennych lingwistycznych i ich reprezentacji za pomocą zbiorów rozmytych (fuzyfikacja)

• Service

FUZZIFY service

TERM poor := (0, 1) (4, 0) ;

TERM good := (1, 0) (4,1) (6,1) (9,0);

TERM excellent := (6, 0) (9, 1);

END_FUZZIFY



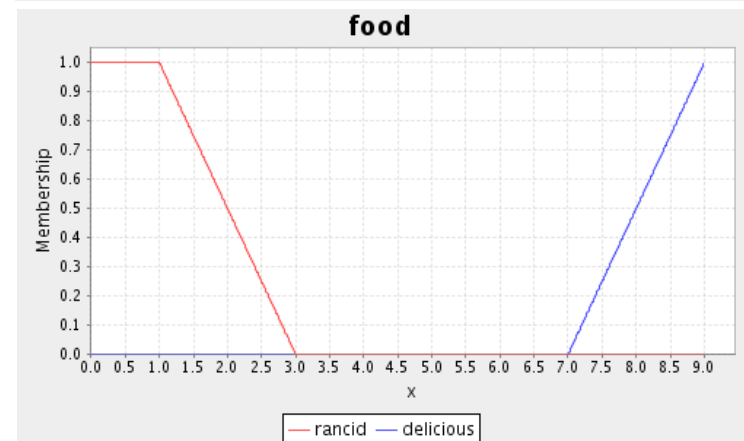
• Food

FUZZIFY food

TERM rancid := (0, 1) (1, 1) (3,0) ;

TERM delicious := (7,0) (9,1);

END_FUZZIFY

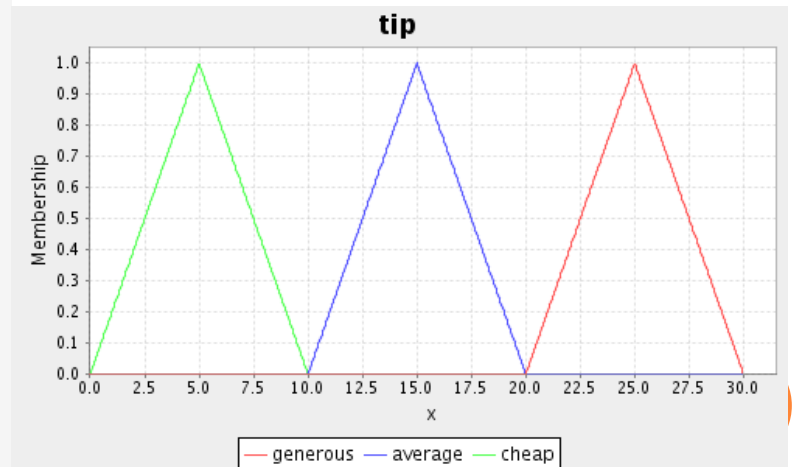


LOGIKA ROZMYTA, C.D.

- Definicja zmiennej wyjściowej tip wraz z jej zbiorami rozmytymi oraz parametrów procesu defuzyfikacji (przekształcenia wartości funkcji przynależności na etykietę lingwistyczną z jej sugerowaną wartością – decyzja)
 - Tip

DEFUZZIFY tip

```
TERM cheap := (0,0) (5,1) (10,0);  
TERM average := (10,0) (15,1) (20,0);  
TERM generous := (20,0) (25,1) (30,0);  
METHOD : COG; // Use 'Center Of Gravity'  
DEFAULT := 0;  
// Default value is 0 (brak pasującej reguły)  
END_DEFUZZIFY
```



LOGIKA ROZMYTA, C.D.

- Definicja reguł wnioskowania oraz interpretacji operatorów AND, OR i dodatkowych parametrów:

RULE 1 : IF service IS poor OR food IS rancid THEN tip IS cheap;

RULE 2 : IF service IS good THEN tip IS average;

RULE 3 : IF service IS excellent AND food IS delicious THEN tip is generous;

AND : MIN; // OR : MAX, aby spełnić prawa DeMorgana

ACT : MIN; // Definicja metody aktywacji reguł, określa brany pod uwagę stopień przynależności zmiennych

ACCU: MAX; // Definicja metody akumulacji; jeżeli kilka metod jest aktywowanych, wynikiem jest ich wartość maksymalna

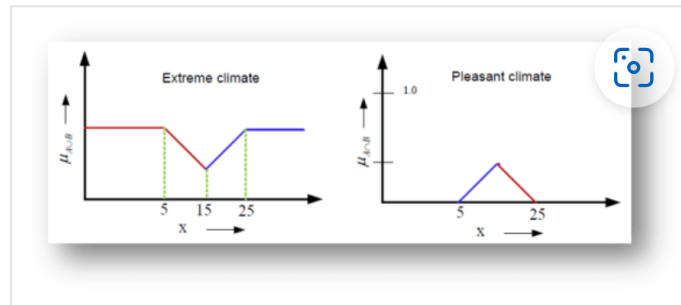
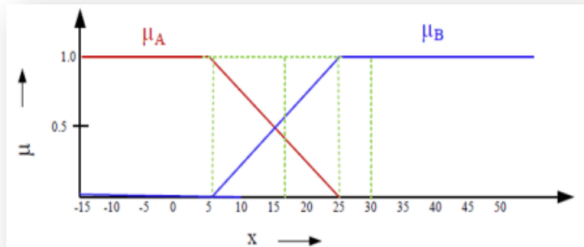


LOGIKA ROZMYTA, C.D.

- A – zimny klimat (μ_A - funkcja przynależności)
- B – gorący klimat (μ_B - funkcja przynależności)

Extreme climate: $\underline{A} \cup \underline{B} = \max(\mu_A(x), \mu_B(x))$

Pleasant climate: $\underline{A} \cap \underline{B} = \min(\mu_A(x), \mu_B(x))$



Union and intersection of fuzzy sets

LOGIKA ROZMYTA, C.D.

○ Przykład wnioskowania:

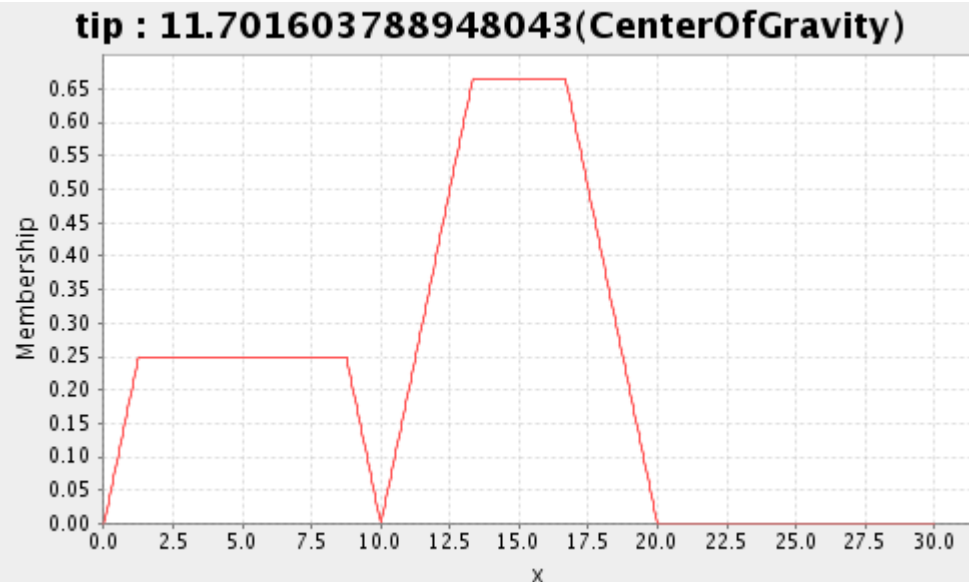
```
fis.setVariable("service", 3); fis.setVariable("food", 7);
```

Output value: 11.701603788948043 (dla centrum masy)

1 (0.25) if (service IS poor) OR (food IS rancid) then tip IS cheap

2 (0.6666666666666666) if service IS good then tip IS average

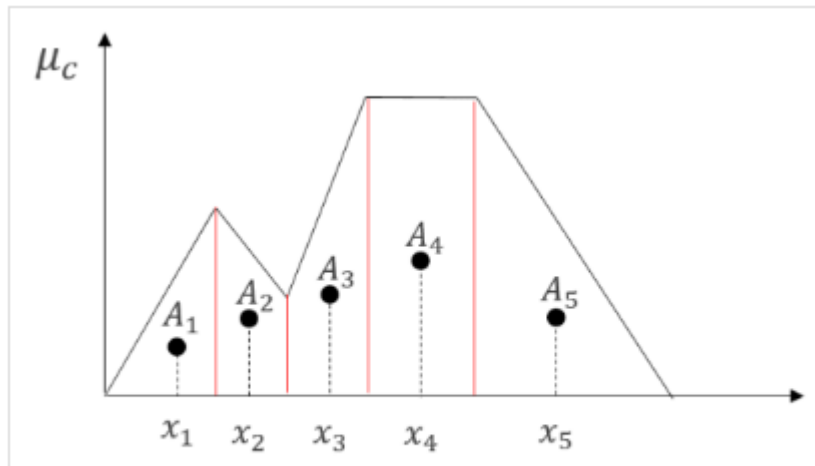
3 (0.0) if (service IS excellent) AND (food IS delicious) then tip IS generous



LOGIKA ROZMYTA, C.D.

- Funkcje defuzyfikacji:
 - Center of Gravity (CoG)

$$x^* = \frac{\sum_{i=1}^n A_i \cdot x_i}{\sum_{i=1}^n A_i}$$



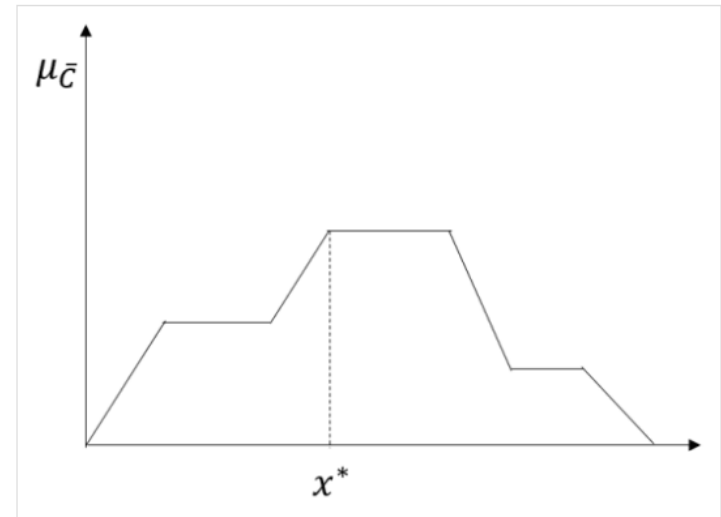
Aggregated fuzzy output (after division)



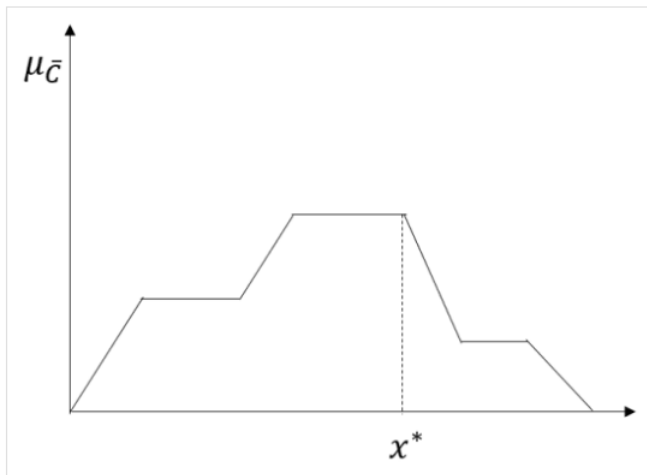
LOGIKA ROZMYTA, C.D.

○ Inne funkcje defuzyfikacji:

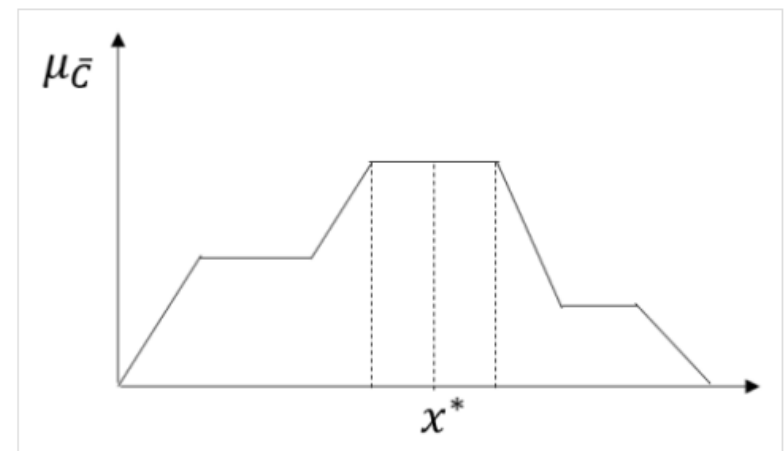
- First of Maxima (FOM)
- Last of Maxima (LOM)
- Middle of Maxima (MOM)



First of maxima



Last of maxima

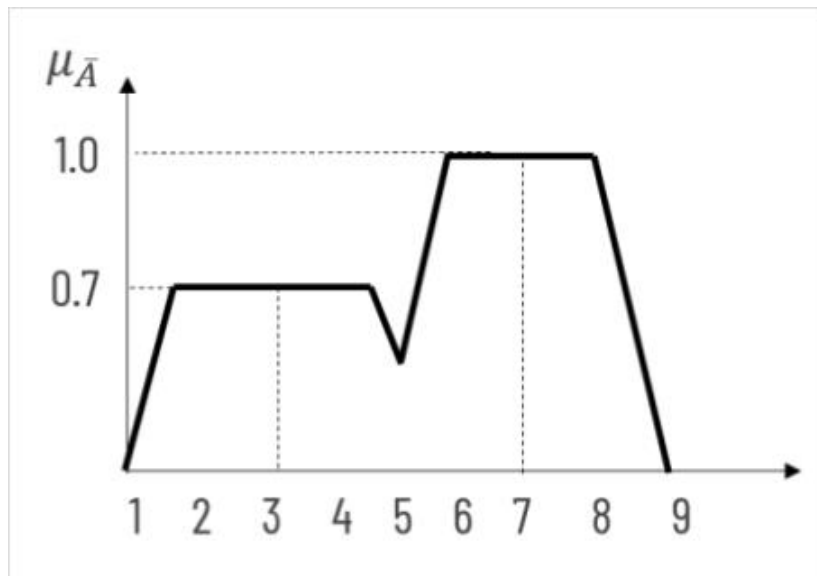


Middle of maxima

LOGIKA ROZMYTA, C.D.

- Inne funkcje defuzyfikacji:
 - Weighted average method

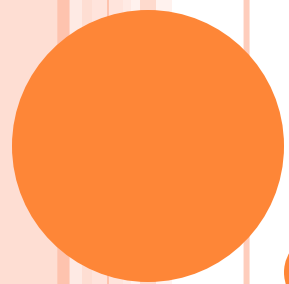
$$x^* = \frac{\sum_{i=1}^n \mu_{\bar{C}_i}(x_i) \cdot (x_i)}{\sum_{i=1}^n \mu_{\bar{C}_i}(x_i)}$$



$$x^* = ((0.7 \times 3) + (1.0 \times 7)) / (0.7 + 1.0)$$

$$x^* = 5.941$$





ZAPIS/ODCZYT DANYCH

WYKORZYSTANIE SERIALIZACJI (JSON)

- Dane do serializacji

```
[Serializable]
public class MyClass
{
    public int level;
    public float timeElapsed;
    public string playerName;
}
```

```
MyClass myObject = new MyClass();
myObject.level = 1;
myObject.timeElapsed = 47.5f;
myObject.playerName = "Dr Charles Francis";
```

- Serializacja

```
string json = JsonUtility.ToJson(myObject);
{"level":1,"timeElapsed":47.5,"playerName":"Dr Charles Francis"}
```

- Deserializacja

```
myObject = JsonUtility.FromJson<MyClass>(json);
```



WYKORZYSTANIE SERIALIZACJI (JSON), C.D.

- Typowe zastosowanie

[Serializable]

```
class GameData { ... }
```

- Zapis ustawień gry

```
private void SaveGameData() {  
    string dataAsJson = JsonUtility.ToJson(gameData);  
    string filePath =  
        Application.dataPath + gameDataProjectFilePath;  
    File.WriteAllText(filePath, dataAsJson);  
}
```



WYKORZYSTANIE SERIALIZACJI (JSON), C.D.

- Ładowanie ustawień gry

```
private void LoadGameData() {  
    string filePath =  
        Application.dataPath + gameDataProjectFilePath;  
    if (File.Exists(filePath)) {  
        string dataAsJson = File.ReadAllText(filePath);  
        GameData loadedData =  
            JsonUtility.FromJson(dataAsJson);  
    }  
    else {  
        Debug.LogError("Cannot load game data!");  
    }  
}
```

INTEGRACJA Z SQLITE

- SQLite – plikowa baza danych (plik do 140TB); implementacja standardu SQL (widoki, klucze obce, transakcje) dystrybuowana jako biblioteka (licencja public domain);
- Ściągnij binaria SQLite dla Twojego systemu operacyjnego (sqlite3.def, sqlite3.dll)
- W folderze Assets utwórz podfolder Plugins; skopiuj do niego sqlite3.def, sqlite3.dll
- Do podfolderu Plugins skopiuj również Mono.Data.Sqlite.dll z C:\Program Files (x86)\Unity\Editor\Data\Mono\lib\mono\2.0
- W swoim skrypcie dodaj następujące przestrzenie nazw: Mono.Data.Sqlite; System.Data, System
- Możesz utworzyć bazę korzystając z klienta konsolowego, graficznego lub w ramach skryptu



INTEGRACJA Z SQLITE, C.D.

```
string connString = "URI=file:" + Application.dataPath + "/Resources/PlayersScore.db";
IDbConnection conn = new SqliteConnection (connString);
IDbCommand command = conn.CreateCommand ();
conn.Open ();

string sql = "CREATE TABLE IF NOT EXISTS players(name Varchar(20) PRIMARY KEY, score INT)";
command.CommandText = sql;
command.ExecuteNonQuery ();

sql = "INSERT INTO players values('Bolek', 12)";
command.CommandText = sql;
command.ExecuteNonQuery ();

sql = "SELECT * FROM players";
command.CommandText = sql;
IDataReader reader = command.ExecuteReader ();
while (reader.Read ()) {
    print ("Player name: " + reader ["name"] + " Score: " + reader ["score"]);
}

reader.Close (); reader = null;
command.Dispose (); command = null;
conn.Close (); conn = null;
```

INTEGRACJA Z SQLITE, C.D.

- Alternatywne rozwiązania:
 - [GitHub - robertohuertasm/SQLite4Unity3d: SQLite made easy for Unity3d](#)
 - [GitHub - praeclarum/sqlite-net: Simple, powerful, cross-platform SQLite client and ORM for .NET](#)



PYTANIA KONTROLNE

- Jak można zrealizować inteligencję w grze (w deterministyczny sposób)?
 - Opisz kluczowe elementy drzewa decyzyjnego
 - Opisz kluczowe elementy drzewa zachowań
 - Opisz kluczowe elementy maszyny stanów
 - Opisz kluczowe elementy GOAP
-
- W jaki sposób można w Unity zapisywać/odczytywać dane?

